

lab6 实验报告

思考题

Thinking 6.1

Q:

如果不关闭自己不会用到的管道端会怎么样？

A:

不关闭无用端会破坏管道关闭检测机制，导致进程无法正确判断对方是否已终止，从而引发资源泄漏、阻塞或异常。

Thinking 6.2

Q:

示例代码中，父进程操作管道的写端，子进程操作管道的读端。如果现在想让父进程作为“读者”，代码应当如何修改？

A:

只需根据fork的结果，交换父进程和子进程应当执行的操作。

Thinking 6.3

Q:

MOS 中的管道缓冲区大小 PIPE_SIZE 过小有什么影响？缓冲区大小是怎么影响吞吐和切换效率的？

A:

- 影响：缓冲区过小会导致写进程频繁阻塞，读进程频繁唤醒，增加进程切换次数，降低整体吞吐量。
- 吞吐与切换效率：缓冲区越小，每次写入的数据量受限，写满后必须切换进程进行读取；切换开销增大，吞吐下降。适当增大缓冲区可减少切换次数，提高传输效率。

Thinking 6.4

Q:

上面这个恒等式在所有时刻都成立吗？请给出你的分析过程(若不成立，会在哪些时刻不成立？)，如果不成立，是否会造成误判？

A:

不总是成立。在执行 `pipe_close` 或 `dup` 的中间时刻（如先解除 pipe 映射再解除 fd 映射），引用计数会暂时失衡。这可能造成 `_pipe_is_closed` 误判另一端已关闭，导致进程提前退出或死锁。

Thinking 6.5

Q:

上面这种不同步修改 `pp_ref` 而导致的进程竞争问题在 `user/lib/fd.c` 中的 `dup` 函数中也存在。请结合代码模仿上述情景，分析一下我们的 `dup` 函数中为什么会出现预想之外的情况？

A:

如果在 `dup` 过程中管道的一个进程执行一半发生了中断，只复制了 `fd` 而没有进行后续操作，则对于管道另一个进程来说，此时检查 `pipe_is_close`，则因为 `data` 对应页面的引用和 `fd` 对应页面的引用一样多，误以为另一个已经结束，导致一些不可预料的后果。

Thinking 6.6

Q:

阅读上述材料并思考：为什么在我们的MOS中，系统调用一定是原子操作呢？如果你觉得不是所有的系统调用都是原子操作，请给出反例。希望能结合相关代码进行分析说明。

A:

每次执行系统调用前都会禁用中断再陷入内核，因此系统调用一定是原子操作。

Thinking 6.7

Q:

仔细阅读上面这段话，并思考下列问题

- 按照上述说法控制 `pipe_close` 中 `fd` 和 `pipeunmap` 的顺序，是否可以解决上述场景的进程竞争问题？给出你的分析过程。
- 我们只分析了 `close` 时的情形，在 `fd.c` 中有一个 `dup` 函数，用于复制文件描述符。试想，如果要复制的文件描述符指向一个管道，那么是否会出现与 `close` 类似的问题？请模仿上述材料写写你的理解。

A:

- 可以解决。如果先解除 `p[0]` 的映射，则 `ref(p[0])` 更要小于 `ref(pipe)`，永远不会出现 `ref(p[0]) == ref(pipe)`，不会出现进程竞争问题。
- 会，`fd` 的引用总数总是比 `pipe` 低一些。当管道的 `dup` 进行到一半时，若先映射 `fd`，再映射 `pipe`，就会使得 `fd` 的引用次数的 +1 先于 `pipe`。这样会使得出现 `pageref(pipe) == pageref(fd)` 的情况成为可能。这个问题也可以通过调换两个 `map` 的顺序来解决。

Thinking 6.8

Q:

思考以下三个问题。

- 认真回看Lab5文件系统相关代码，弄清打开文件的过程。
- 回顾Lab1与Lab3，思考如何读取并加载ELF文件。
- 在Lab1中我们介绍了 `datatextbss` 段及它们的含义，`data` 段存放初始化过的全局变量，`bss` 段存放未初始化的全局变量。关于 `memsize` 和 `filesize`，我们在Note1.3.4中也解释了它们的含义与特点。关于Note1.3.4，注意其中关于“`bss` 段并不在文件中占数据”表述的含义。回顾Lab3并思考：`elf_load_seg()` 和

load_icode_mapper()函数是如何确保加载ELF文件时，bss段数据被正确加载进虚拟内存空间。bss段在ELF中并不占空间，但ELF加载进内存后，bss段的数据占据了空间，并且初始值都是0。请回顾elf_load_seg()和load_icode_mapper()的实现，思考这一点是如何实现的？

A:

- 用户程序调用open,包装成fsipc_open,通过IPC发送FSIPC_OPEN请求给文件服务进程(fs进程)，fs进程调用file_open查找文件，分配相应的文件描述符，并通过IPC返回fd和共享页。用户获得fd,并再次通过IPC操作文件。
- 将elf文件先编译成.b文件再反编译成.b.c文件，最后把.b.c文件以char类型数组的形式装载到MOS中，在加载MOS的过程中一起执行elf文件
- 对于bss段中和text&data段共同占据一个页面的部分，就将这一部分置零；对于bss段其它部分，仅使用syscall_mem_malloc分配页面而不映射到任何内容。

难点分析

我认为本次实验应该可以说是除了lab1和lab0之外最简单的一次lab了，几乎没有难点，唯一可能造成理解障碍的地方在于spwan一个进程时不停调用fork和spwan，让人摸不着头脑，以及其中的gettoken这个函数也是惊为天人，一个简单的函数怎么能搞的如此复杂。

实验体会

lab6结束之后，一个简易的mos也总算完工了，还是很有成就感的，一路上也学到了很多，不过这个shell还是太垃圾了，各种意义上的，希望能通过挑战性任务打造一个能用的舒服一点的shell。